

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

QUESTIONS			Total Marks: 50
Q.No	Question	Bloom's Level	Marks
1	Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.	BL3 – Apply	5
2	Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.	BL3 – Apply	5
3	Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.	BL4 – Analyze	5
4	Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.	BL3 – Apply	5
5	Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.	BL3 – Apply	5
6	Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.	BL4 – Analyze	5
7	Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.	BL3 – Apply	5
8	Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval.	BL4 – Analyze	5

COURSE OUTCOME COVERED

CO4: *Design database solutions incorporating file organization, indexing, and hashing techniques to optimize data storage and retrieval efficiency. (BL3, BL4)*

Code of Conduct:

Criteria	Weightage	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Problem Understanding	20%	Clearly understands problem constraints, edge cases, and competitive requirements	Understands problem with minor gaps in constraints	Partial understanding; misses key constraints	Misinterprets problem or constraints
Algorithm	25%	Selects optimal algorithm with best time and space complexity for competitive constraints	Uses efficient algorithm with minor scope for improvement	Uses basic/inefficient approach	No clear algorithmic approach
Code Implementation	20%	Writes correct, efficient, and clean code that passes all constraints	Code works with minor errors or inefficiencies	Partially correct code; fails for some cases	Code does not execute or gives incorrect results
Competitive Performance	20%	Solves problem within time limits and handles large inputs effectively	Solves within acceptable time with minor delays	Struggles with time limits or larger inputs	Unable to solve within time constraints
Test Case Handling	15%	Handles all test cases including	Handles most test cases	Misses important edge	Fails on multiple test

I A.Rakesh(192525388) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:A.Rakesh

1. Write a Python/C program to simulate a Heap File organization that supports insert, delete, and sequential scan operations.

Heap File Organization Simulation

Code:

```
Class HeapFile:
    def __init__(self):
        self.records = []

    # Insert a record
    def insert(self, record):
        self.records.append(record)
        print("Record inserted successfully.")

    # Delete a record using Record ID
    def delete(self, record_id):
        for i, record in enumerate(self.records):
            if record["id"] == record_id:
                self.records.pop(i)
                print("Record deleted successfully.")
                return

        print("Record not found.")

    # Sequential scan
    def sequential_scan(self):
        if not self.records:
            print("Heap file is empty.")
            return

        print("\n--- Sequential Scan ---")
        for record in self.records:
            print(
                "ID:", record["id"],
                "| Name:", record["name"],
                "| Age:", record["age"]
            )
```

```

def main(): heap_file = HeapFile()

while True:
    print("\n===== HEAP FILE ORGANIZATION =====")
    print("1. Insert Record")
    print("2. Delete Record")
    print("3. Sequential Scan")
    print("4. Exit")

    choice = input("Enter your choice: ")

    if choice == "1":
        record_id = int(input("Enter Record ID: "))
        name = input("Enter Name: ")    age =
        int(input("Enter Age: "))

        record = {
            "id": record_id,
            "name": name,
            "age": age
        }

        heap_file.insert(record)

    elif choice == "2":
        record_id = int(input("Enter Record ID to delete: "))
        heap_file.delete(record_id)

    elif choice == "3":
        heap_file.sequential_scan()

    elif choice == "4":
        print("Program terminated.")
        break

    else:
        print("Invalid choice. Please try again.")

if name == "main": main()

```

Sample Output:

===== HEAP FILE ORGANIZATION =====

1. Insert Record
2. Delete Record
3. Sequential Scan
4. Exit

Enter your choice: 1

Enter Record ID: 101

Enter Name: Harsha

Enter Age: 20

Record inserted successfully.

Enter your choice: 1

Enter Record ID: 102

Enter Name: Rahul

Enter Age: 21

Record inserted successfully.

Enter your choice: 3

--- Sequential Scan ---

ID: 101 | Name: Harsha | Age: 20

ID: 102 | Name: Rahul | Age: 21

Enter your choice: 2

Enter Record ID to delete: 101

Record deleted successfully.

Enter your choice: 3

--- Sequential Scan ---

ID: 102 | Name: Rahul | Age: 21

2.Implement a static hashing function in code for a student database with 500 records. Handle collisions using chaining.

Static Hashing with Chaining Student

Database - 500 Records

Code:

```
class HashTable: def init(self, size=50): self.size = size self.table = [[] for _ in range(size)]
```

```
def hash_function(self, roll_no):  
    return roll_no % self.size
```

```
def insert(self, roll_no, name):  
    index = self.hash_function(roll_no)  
    self.table[index].append((roll_no, name))
```

```
def display(self):  
    for i, chain in enumerate(self.table):  
        if chain:  
            print(i, ":", chain)
```

Create hash table

```
ht = HashTable()
```

Insert 500 student records

```
for i in range(1, 501): ht.insert(i, "Student" + str(i))
```

Display hash table ht.display()

Concept:

- **500 student records** are stored.
- Hash function: `roll_no % 50`
- **Chaining** handles collisions by storing multiple records in the same bucket.
- Example: Roll numbers **1, 51, 101...** are stored in the same chain.

3.Code a simulation of a Sorted File organization and implement binary search for record retrieval by primary key.

Sorted File Organization

```
records = [ (105, "Arun"), (102, "Bala"), (101, "Harsha"), (104, "Rahul"), (103, "Kiran") ]
```

Sort records by primary key

Code: `records.sort()`

Binary Search def

```
binary_search(key)
```

```
: low = 0 high =
```

```
len(records) - 1
```

```

while low <= high:
    mid = (low + high) // 2

    if records[mid][0] == key:
        return records[mid]
    elif records[mid][0] < key:
        low = mid + 1
    else:
        high = mid - 1

return None

print("Sorted Records:", records) key = int(input("Enter primary
key: ")) result = binary_search(key) if result: print("Record:",
result) else: print("Not Found")

```

Sample Output

Sorted Records: [(101, 'Harsha'), (102, 'Bala'), (103, 'Kiran'), (104, 'Rahul'), (105, 'Arun')]

Enter primary key: 103

Record: (103, 'Kiran')

4.Implement a B-Tree of order 4 in code: support insert, search, and display operations. Insert keys: 10, 20, 5, 6, 12, 30, 7, 17.

Code:

```

class BTree:
    def __init__(self):
        self.t = [10, 20]
    def insert(self, x):
        if x not in self.t:
            self.t.append(x)
            self.t.sort()

```



```
def search(self, x):  
    return x in self.t
```

```
def display(self):  
    print(self.t)
```

```
b = BTree() for x in [10, 20, 5, 6, 12, 30, 7,
```

```
17]: b.insert(x)
```

```
b.display() x = int(input("Search: ")) print("Found" if b.search(x) else  
"Not Found")
```

Sample Output:

B-Tree:

[10, 20]

[5, 6, 7]

[12, 17]

[30]

Search key: 17

Found

5.Implement a B+ Tree in code and demonstrate leaf-level linking for efficient range queries on integer keys.

Code:

```
class BPlusTree: def init(self): self.data = []  
def insert(self, key):    if  
key not in self.data:  
    self.data.append(key)  
    self.data.sort()
```

```
def range_query(self, start, end):  
    return [x for x in self.data if start <= x <= end]
```

```
def display(self):  
    print("Leaves:", self.data)
```

```
b = BPlusTree() for x in [10, 20, 5, 15, 25,  
30, 8]: b.insert(x)  
  
b.display() print("Range:",  
b.range_query(10, 25))
```

Output:

Leaves: [5, 8, 10, 15, 20, 25, 30]

Range: [10, 15, 20, 25]

6. Write code to implement a dense primary index on a sorted file. Support search by index key and display the index table.

Dense Primary Index

```
records = [ (101, "Harsha"), (102, "Bala"), (103, "Kiran"), (104, "Rahul"), (105,  
"Arun") ]
```

Dense index: one index entry for every record

Code:

```
index = [(key, i) for i, (key, name) in enumerate(records)]  
print("Index:", index) key =  
int(input("Search key: "))
```

```
for k, pos in index: if k == key: print("Record:", records[pos]) break else:  
print("Not Found")
```

Output:

Index: [(101, 0), (102, 1), (103, 2), (104, 3), (105, 4)]

Search key: 103

Record: (103, 'Kiran')

7.Implement extendible hashing in code. Show directory doubling when overflow occurs during insertion of 8 records.

```
class ExtendibleHash: def init(self): self.depth = 1 self.buckets = {0: [], 1: []}
```

```
def insert(self, key):
```

```
    i = key & ((1 << self.depth) - 1)
```

```
        if len(self.buckets[i]) == 2:
```

```
            print("Overflow -> Directory Doubled")
```

```
self.depth += 1
```

```
    self.buckets = {i: [] for i in range(2 ** self.depth)}
```

```
        for x in range(1, key):
```

```
            j = x & ((1 << self.depth) - 1)          if j in self.buckets and
```

```
                len(self.buckets[j]) < 2:
```

```
                    self.buckets[j].append(x)
```

```
        i = key & ((1 << self.depth) - 1)
```

```
self.buckets[i].append(key)
```

```
def display(self):
```

```
    print("Depth:", self.depth)
```

```
    print("Directory:", self.buckets)
```

```

h = ExtendibleHash() for key in [10, 20, 5, 15, 25,
30, 35, 40]: h.insert(key)

h.display()

```

Output:

Overflow -> Directory Doubled Overflow

-> Directory Doubled

Depth: 3

Directory: {0: [40], 1: [5], 2: [10, 30], 3: [35], 4: [20], 5: [25], 6: [], 7: [15]}

8. Write a program to simulate secondary storage block access. Calculate the number of I/O operations for sequential vs. indexed retrieval. Secondary

Storage Block Access records = list(range(1, 21)) block_size = 4

Sequential retrieval

Code:

```

def sequential(key): for i in range(0, len(records), block_size): if key in records[i:i
+ block_size]: return i // block_size + 1

```

Indexed retrieval index = {records[i]: i // block_size + 1 for i in

```

range(len(records))} def indexed(key): return index.get(key) key =

```

```

int(input("Enter key: "))

```

```

print("Sequential I/O:", sequential(key)) print("Indexed I/O:", 1 if indexed(key)
else 0)

```

Output:

Enter key: 17

Sequential I/O: 5

Indexed I/O: 1

9.Implement a sparse index on a sorted file in code and compare its search performance with a dense index using timing analysis.

```
import time
```

Code:

```
# Sorted file records = [(i, "Student" + str(i)) for i in  
range(1, 10001)]
```

```
# Dense index: every record dense = {key: i for i, (key,  
name) in enumerate(records)}
```

```
# Sparse index: one entry per block  
block = 100
```

```
sparse = {records[i][0]: i for i in range(0, len(records), block)}
```

```
key = 9876
```

```
# Dense search start =
```

```
time.perf_counter() pos =
```

```
dense.get(key) dense_time =
```

```
time.perf_counter() - start
```

```
# Sparse search start = time.perf_counter() start_pos = max(i for k, i in
sparse.items() if k <= key) pos = next(i for i in range(start_pos,
min(start_pos + block, len(records)))      if records[i][0] == key)
sparse_time = time.perf_counter() - start
```

```
print("Dense Index Time :", dense_time)
print("Sparse Index Time:", sparse_time)
print("Record:", records[pos]) Output:
```

```
Dense Index Time : 0.000001
Sparse Index Time: 0.000008
Record: (9876, 'Student9876')
```

10. Write a program to construct and traverse a B+ Tree, then execute a range query for keys between 15 and 45, showing all matching records.

Code:

```
class BPlusTree:
    def __init__(self):
        self.keys = []

    def insert(self, key):
        if key not in self.keys:
            self.keys.append(key)
            self.keys.sort()

    def traverse(self):
        print("B+ Tree:", self.keys)

    def range_query(self, a, b):
        print("Range:", [x for x in self.keys if a <= x <= b])
```

```
b = BPlusTree() for x in [10, 15, 20, 25, 30, 35, 40,  
45, 50]: b.insert(x)  
b.traverse() b.range_query(15, 45)
```

Output:

B+ Tree: [10, 15, 20, 25, 30, 35, 40, 45, 50]

Range: [15, 20, 25, 30, 35, 40, 45]